

Lec 03 Optimizers - Part A

Lec 03 Optimizers - Part A

Lecture	03 / 63 + intro
Duration	31:03
Captions	English (auto-generated)
Words	4,835
Video	VevYEYDAm_8
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone. Welcome to the lecture series on fundamentals of generative AI and the large language models. We are in the week one. In the previous session, we have started with generative AI fundamentals and then we have also looked into the different activation functions and then the loss functions that are used in neural network or deep neural networks. In this particular session, we will see one of the most important concepts called optimizers. What exactly is the role of an optimizer and optimizers for training and deep neural network?

[00:38] So, let us look into this diagram. From this diagram, we have seen again from our conventional multi-layer perceptron or deep learning course that we have an input. That input is a passed to the neural network. Here, what neural network that we have considered is a simple multi-layer perceptron. Now, we we know from the fundamentals that every of the neuron is connected to every other neuron and that connection is being made with the help of and on that connection, we will have the weights associated with it. So, initially, these weights are very randomly chosen. Okay?

[01:16] So, because of this randomness in the weight and the random weights that we have assigned, whatever the prediction made by the model, definitely that predicted

value will not be a correct one. Why? Because the initial weights are random weights and they may not be a correct weights. So, once we get the predicted output, so input is passed to a neural network and the neural network, it will go through from layer to layer and we have weights and associated bias. Now, and these weights and bias are the parameters here and those parameters are randomly initialized. Because of this randomness, the initial output prediction made by the model will be a very will be incorrect prediction.

[01:59] So, once we have this incorrect prediction, the next step is we are going to calculate the loss. Now, uh to for this particular loss, how is that you will tell that the prediction made by the model is wrong? We already have the ground truth with us. We already have the label with us. And whatever the model that has a predicted, we are going to compare these two. So, here, for example, if the actual answer is one, and if the model has a predicted 0.2, definitely the loss is high. Or if the model has a predicted 0.95, then the loss is uh less. So, what exactly loss will tell us? The loss will tell how uh wrong the model is. This is what the loss is going to give us the information. Okay. Now, once the loss is

[02:46] high, what is our aim? Our aim is to reduce this loss. When the um if we reduce this loss, what is the you know, intuition behind? Is it not the ground truth and then the predicted value will be closer or will be similar? Then only the loss value will be less, right? Uh you you will get this concept. You you understood this concept. So, now, you are you are saying that the loss is high. If the loss is high, our aim is to reduce this loss. If our aim is to reduce this loss, what is that we do? We compute the gradients. So, here is the step where we are computing the gradients. How is that uh What is the gradient actually? It is the derivative of loss with respect to weight. So, what is the uh this

[03:33] particular step will tell us? Now, there are weights, there is an input input with weights, bias, some operation is happening, and you're passing it to to layer to layer activation functions, and some output we have got. So, is it not uh um the loss is having some impact because of weights also. So, this derivative of loss with respect to weight, what we are computing the gradients. So, this will tell how each weight has contributed to the loss.

[04:04] Now, this gradient, based on change in the weight, it will have an impact on the loss. If we increase the weight, accordingly there will be an impact in the loss. If you decrease the weight, accordingly there is an impact on the loss. So, is it not this gradient is actually telling us how each weight has contributed to the loss. Now, once we compute the gradient, so it also gives us the direction of weight update and the amount of influence that weight has on the loss. Next step is optimizer.

[04:38] What is the role of an optimizer? The role of an optimizer is this is what is our topic of this particular session. The role of optimizer is it is going to use these

gradients. Why? Ultimately, aim is to update the weights. By updating the weights, what is your aim? So that your model gives correct prediction. Gradually, the prediction should improve. So, the role of an optimizer is to use the gradient to update the weights in the right direction so that the model prediction gradually improves. Therefore, the primary objective of optimization is to minimize the loss function. Now, if that is the case, what is the mathematical formula or a definition for an you know, minimizing this loss or weight updation is W_{new} .

[05:34] New weight is old weight minus learning rate. You see, derivative of loss with respect to weight. Optimizer uses these gradients to update the weights in the right direction so that ultimately this predicted output will be as close as the ground truth. That's an ultimate aim. Now, when you take this gradient, that is derivative of loss with respect to weight, can be either a positive value or it can be a negative value. It can be greater than zero or it can be less than zero. Either it can be positive or negative. So now, once again the fundamental, why negative symbol here? Why negative sign? Minus sign. So we subtract the gradient because we ultimately want to move in the direction where our loss decreases.

[06:23] So because of this it is a negative one. So here you need to let's take one simple example and then understand. W_{new} is W_{old} minus learning rate derivative of loss with respect to weight. Now, we we have seen from the previous that this can be either a negative value or a positive value. See here. Suppose if the W_{old} if this W_{old} value is a five and if the learning rate this is a learning rate which is 0.1 and this derivative of loss with respect to weight is a minus two.

[07:03] I'm just giving one example to show how the weights updates weight gets updated. Substitute the values into each of the this. W_{old} is a five minus learning rate is 0.1 derivative of loss with respect to weight is minus two. Substitute, calculate this. We are going to get 5.2. So is it not the weight has increased here? Or is there any relation are you understanding? So optimizer uses the gradients. So if the gradient is a negative, if the gradient is a negative value, is it not the weight is increased? What do you mean by weight has increased?

[07:42] Earlier weight is five. Now the weight is 5.2. Uh in the similar way, look at the other example uh here. Here the derivative of loss with respect to weight is a positive. So, earlier old weight was five, now you have reduced the weight. Right? Decreased the weight. So, from this, what are we understanding? So, when the gradient is negative, subtracting a negative value makes the weight increase. So, when the weights are positive, the optimizers uh reduces the um you know, weights. When gradient is a positive, it reduces the weight. When gradient is a negative, it is increasing the weight. So, from this, are we understanding that there is some relation with this uh gradients? Right? So, optimizer do

[08:28] not randomly change the weight. It [snorts] will always uses the gradient of

the loss uh function, and then it will decide whether we have to increase the weight or decrease the weight. So, since the topic is all about the optimizer, we have understood what is the role of an optimizer in a neural network or in a deep learning. Optimizers are the one which will mainly helpful in optimizing those weights. And what do you mean by optimizing the weights? Either increasing or decreasing. Here our aim is either increase the weight or decrease the weight. We are not bothered, but ultimately aim is loss should be minimized. That is the role of an optimizers.

[09:10] If that is the case, what all different optimizers we have? The very first optimizer uh that we have is Okay, these are the three set of optimizers that we have. First is a stochastic gradient descent, then mini batch stochastic gradient descent, and the gradient descent. Okay, let us uh see what is uh the meaning of each of these. Now, uh can you able to understand from this uh diagram? These are the training samples.

[09:37] So, whatever we are seeing, these are the training samples. Now, these training samples has to be passed to model and then model will predict and then you'll be calculating the error. Now, when you are passing these samples, in what all different ways you can pass? Can I pass only one sample at a time? Answer is yes. So, we are passing the first sample. We have passed it to a neural network. The model has given the prediction. And for this sample, there is an weight I mean there is an ground truth. You're going to calculate the loss or an error by comparing this prediction made by the prediction made by this particular sample and it's ground truth. Then, after you have calculated, you are computing the

[10:24] gradient. You are coming back to update the weights. Next, you will you will now pass the second sample. Second sample is being passed to a neural network. Now, for this second sample, you will get a prediction P2. Now, for this sample, there is a ground truth Y2. You are going to calculate the error between these two and then you will come back and then you will update all the weights that are there in the neural network. Then, you will pass the third sample.

[10:56] When you pass the third sample, you will get third prediction. You will have its ground truth Y3. Again, you will calculate the error. Then, you will go back. You will update the weights. After you update the weights, you will pass the next sample. Are you understanding how we are computing how we are updating the weights here? After every sample that you are passing, you are calculating the error or loss. These are oftenly interchanged. Okay, you will calculate the error and then you will come back, back propagate. You will use optimizers, you will update the weights. Now, is it not something now if you do this, it is called a stochastic gradient descent. Look at this stochastic gradient diagrammatically visualization.

[11:46] Ultimately, our aim is to reach it to this global minima because aim aim is to reduce the loss or an error and aim is to get the error value as close as a zero. This

is our ultimate aim. If this is our aim, now you have passed first sample. By passing the first sample, based on the weights, based on the prediction, based on its ground truth, based on this gradients what we have computed, let us assume the weight has increased from the previous weight to to the current weight. Weight has increased. How does it? We have seen it in the previous one. How weights increases, decreases based on the gradient, we have seen in this example, right? Now, when you look into when you pass the first sample, let us

[12:35] assume the weight has increased. Okay? Now, you have passed this second sample. By passing the second sample, for this sample and the weight that got updated, you have computed its error or the loss and then you have back propagated. Again, you have changed the weights. By changing the weights this time, let us assume you have decreased the weights. Okay. Again, you have passed the third sample. For third sample, again based on the weights, based on its ground truth, prediction, you have calculated the error, again back propagated. This time, the weight increased. Is it not too many times you are oscillating?

[13:16] Is it not too much of oscillation? Weight increase, weight decrease. Weight increase, weight decrease. Weight increase, weight decrease. Is it not too much of movement here? So, that is uh is stochastic gradient descent. I'm not saying it's a drawback or anything. I'm just saying this is what is the stochastic gradient descent where ultimate aim is to update this weight. Whether you are updating the weight by passing one sample at a time, if yes, then it is stochastic gradient descent.

[13:44] What is mini batch? It's there in the word itself. Mini batch, which means you are now not passing only one sample, you are passing a smaller set of So, you are now taken two. Here for this example, I've taken two, but otherwise it's an hyperparameter. We have to specify how many samples we should consider. Now, what is the meaning of this? You are passing the first sample to a neural network, you will get the prediction one.

[14:12] For this prediction, you have an error Y_1 . I mean, ground truth Y_1 . You are calculating the error. Error will be E_1 . Then, you will pass the second sample. You are not back propagating. Hold on. So, you you for this the second sample, you are getting a second that prediction. It has its associated ground truth Y_2 . With the help of predicted and ground truth, you are calculating error two. Now, you will add error one, error two. You will now combine these errors, then you will back propagate to update the weights. So, you see previously in stochastic gradient, after every sample, you updated the weights, but here after two samples, after mini batch in this example two, but otherwise in general in mini batch,

[14:58] that mini batch will have some subset. So, you will pass all of these samples, you will get so many errors, you will add up all those errors, then you will come back to update the weights. Hope this is clear to you, right? So, this is uh now, when

you do this, is it not definitely this is reduced? Uh Uh, see the difference between these two. Right? So, definitely mini batch is a better when comparing to stochastic gradient descent. However, all these uh depends on the kind of data that we are using. So, there is no thumb rule that we should use only mini batch or only stochastic. No. Based on your problem statement, you can accordingly change these and then see for which parameter we are getting for which hyperparameter

[15:47] you're getting a good performance. Now, the third one uh next, after you complete this, you will now pass the next two samples. How do you pass? First, you will pass this sample. You will pass it to the neural network. You will get that prediction. You will have that ground truth. You will now calculate the error. You will get E3. Then, you will pass this sample. You will get P4 predicted ground truth Y4. You will get error four. You will combine these two, calculate these two, add these two errors. Then, you will back propagate.

[16:22] When you back propagate, you will update the weights. Once you update the weights, you will now pass this sample. Why this sample? Now, you're left out with only one sample in the training set. So, whatever the last a few set you have, which is not equivalent to that mini batch size that you have specified, which means those are all left out of samples. All those samples now you'll again pass this. You will calculate P5. There is a ground truth. You will calculate error.

[16:49] Since there is only one sample after this error, you will back propagate. So, this is called you know, mini batch stochastic gradient descent. And the third one we have a gradient. You see, these are the training samples that we have. Now, we are taking all of those samples, all the training samples we are taking, and then we are calculating the error, and then we are back propagating. What does it mean? We are taking the first sample, we are getting a P1, Y1, error one. We are passing the second P2, Y2. I hope you are understanding when I say P2 predicted value, Y2 is it's associated ground truth error. P3, Y3, error three.

[17:34] P4, Y4, error four. P5, Y5, error five. Now, you will add up all of these errors, you will get a final error E. That you will calculate that you will come back to update the weights. Not after every sample you are back propagating and you are updating. You are passing all of the training samples and then you are calculating uh you're updating these weights. So, this is the initial set of optimizers, which are gradient descent, stochastic mini batch stochastic gradient descent, and stochastic gradient descent. And uh we have These are the initial set of optimizers. Then, let us look into one of the most uh from here on whatever optimizers that we discuss are the most important optimizers.

[18:30] » [snorts] » Now, let us see stochastic gradient descent with the momentum. Now, for that let us try to understand this SGD is stochastic gradient descent, which we have seen in the previous, where after every of the sample we are updating the

weights. So, because of this there is too much of movement, right? There is up, down, up, down. There is too much of zigzag movement. So, this is what is the problem with the stochastic gradient descent. Is it not?

[19:00] In stochastic gradient descent, this value derivative of loss with respect to weight will be one. Why one? Because uh here okay, let's go back and then understand this concept. You see here when you talk about stochastic gradient descent, then this is based on only one error. So, when you take mini batch stochastic gradient descent, this will be based on your K values, whatever is that K samples. And in gradient descent, it is all the samples we have considered. So, this is what is one that you are seeing in this particular slide.

[19:37] This one here is in a normal stochastic gradient that we take. The weight update uses only the current gradient. And it is going to update the weight. So, for the first one, if the weights increases, if the gradient is a negative, the weight increases. Next sample you have passed, then the gradient has gradient is positive. If the gradient is a positive, you are reducing the weight. Again, you have passed the next sample. Gradient is a negative, then you have increased the weight. So, because of this there is too much of zigzag here. Too much of time it will take to converge, to come down to an you know global minima. So, what we are doing, we are using a concept called momentum here. What is momentum? Just to see,

[20:29] momentum is nothing but a velocity, speed. So, velocity here. So, can we can we think in this direction? If I am going in a flat surface, so definitely my speed will be very fast. My walking will be very fast. But when I'm walking in a steep direction, so definitely I'll be going a little slow. That is one analogy. Second analogy, you see, now if the gradients are positive for a long time, that means for a current iteration gradient is positive.

[21:03] For the previous gradient is positive. For its of the gradient is positive. If consistently for from quite some time, if the gradients are all positive, can I move little fast? Can I walk little faster towards the global minima? Is this intuition you are getting? If it is yes, then is it not we are minimizing too much of zigzag? Is it not we have a little smooth, you know, direction of movement? In the gradient descent SGD, the main problem is there is too much of zigzag.

[21:40] Why too much? After every sample, you are updating the weights. So, based on the gradient, if gradient is negative, weight is increasing. If the gradient is positive, weight is decreasing. So, after every sample, there is no control that gradient can be positive or negative. So, there is too much of zigzag in SGD. So, that we don't want. We want little smoother movement. Then, some change we have to do. What is that the change? Can you see here?

[22:08] The weight update formula is $W_{\text{new}} = W_{\text{old}} - \text{learning rate} \cdot \text{velocity}$. Look at this velocity term VT . What is this a VT ? VT is a given by some you know, constant β VT

minus one plus one minus beta GT. GT here is once again gradient, derivative of loss with respect to weight. So, is it not we have VT? You can even imagine in case if GT is confusing to you. Can we write in this way? So, this entire VT is again W_{new} is equal to W_{old} minus learning rate and this VT. This VT whole if you substitute here, you can visualize how the equation will be. Now, what are we trying actually? What is this formula? Why did we write this formula? Is there any meaning this formula is trying to give it give us? This is

[23:10] you know, exponentially weighted moving average. Now, exponentially weighted moving average. What exactly is exponentially? Now, you are also considering you know, the older gradients and you are also considering the current gradient. So, it is just not one gradient current gradient we are considering to update the weights, which means the recent gradients also should be considered, but give more importance to the recent gradients while old gradients are still remembered, but with a decreasing importance. This is the definition of exponentially weighted or the meaning of exponentially weighted.

[24:01] » [snorts] » But, probably you may not get the actual intuition. Let us take one example. What is the example? Take this small intuition. What is ultimately the formula is this, right? And this is a gradient. Let us assume the gradient what we have obtained is ϕ . Okay, this is the gradient and beta value is the momentum term, which we have considered it as 0.9. This is the standard value that we usually fix. And since it is the first time you are calculating the prediction and then the error, so definitely V_0 since it is the first V_0 value will be zero. Just substitute the values in this and then see what is that we are getting. Is it not we are getting V_1 ?

[24:44] How are we getting this V_1 ? 0.9 is substituted. This $T-1$ is the previous. This is the first one, so zero. $1 - 0.9$ is 0.1 and the current gradient let us assume, is a positive value. So, which is a positive. Just substitute the values in this and the velocity term, you look at this velocity term. What we got is 0.5. Okay? Now, for the next iteration, we have passed set of samples or we have passed the samples here. And for those samples, the gradient, this is the gradient that we have obtained. And look at the formula that we are using. The formula is beta is 0.9. $V_t - 1$. When you are doing for a time step two, we have to consider V_1 . Plus $1 - 0.9$ is 0.1 and the current gradient is 6. So, substitute the values in this

[25:41] formula and then see what is the value we have obtained. V_2 , we have obtained this. Okay, still have not explained what is the intuition behind. Now, the third, you have now seen at the third instance where the gradient what you have obtained is, once again, you have obtained a positive gradient. And you see how we are considering 0.9. What was my previous gradient? How was my previous? So, are we not taking V_2 ? V_2 is the previous step, right? V_2 plus 0.1 and its gradient, current gradient seven. So, previous V_2 is so much. You have added, you have substituted the values in this and then you have obtained this velocity. So, now forget all these

calculations. Just to look at only V_1 , V_2 , V_3 . What are we

[26:32] understanding from this? And try to see the relation between V_1 and G_1 , V_2 with G_2 , V_3 with the G_3 . Try to see the relation here. What are we understanding? If the gradients are moving in a similar direction, what is similar direction here? It is a going in a positive direction. Similar direction or pointing in the same direction, then is it not the velocity is increasing? Velocity was initially 0.5, now it is 1.05, now it is 1.645.

[27:03] So, is it not it is increasing the velocity, which means what? When it is a flat surface, I need not have to take a smaller steps. I can walk fast, right? So, when when you are trying to update the weights, now when I know that the gradients from the past the gradients are going in the same direction, then is it not I I can move a little more faster because from the previous I've already obtained some information. From its a previous we've already obtained the information. So, the previous gradients we are taking the help to update the weights here. I mean, sorry, not weights, to update the speed velocity term. Yes, ultimately we are once you have this VT, you're going to sub the values in this formula and then

[27:52] see the impact of the weights here. What we are trying to understand here is SGD, there is too much of movement. Zigzag, too much of zigzag movement. We don't want so complicated. We want a smooth transition. When we want a smooth transition, which means what? When I am going in a same direction, I can be a little faster. So, that there is no too much of zigzag here. So, you see when the velocity when the gradients are constantly moving in a similar direction, then I can be faster.

[28:31] Velocity is automatically increasing. Now, this is what is stochastic gradient descent with the momentum. Now, you can ask what about what if the gradient becomes negative? So, if the gradient becomes a negative, you see, there is no too much abrupt change. When the gradient is a negative, you are once again taking 0.9 formula. This is the previous Previous is 1.645 $1 - 0.9$ is 0.1 and the gradient is -4. When you see here, you see there is a some speed has reduced. From this to this, although there is a negative, there is no too much a change. There is a small change. So, can you now visualize this? There will be a smooth transition rather than abrupt transition. Now, we will have smoother transition.

[29:28] So, this is what is stochastic gradient descent with the momentum. Now, I would like to reiterate this concept. Can you see this? Exponentially weighted moving average. What is the meaning of it? The meaning is written here. Recent gradients are given more importance, while older gradients are still remembered, but decreasing importance. Okay? Now, this is what is exponential. We are still considering the previous one, but only for a certain you know, with a certain limitations you are considering the previous. To understand that, let us take one more example in upcoming slides. So, this is all about Okay, and this is a negative If at all if the gradients are

negative, we are going to have this

[30:18] particular smooth curve. So, with this, I'm coming to end of this particular session. And in this session, what we have realized what we have understood is we have started with understanding the optimizers. What is the role of an optimizer? And we have seen only four different optimizers. That is gradient descent, stochastic gradient descent, mini batch stochastic gradient descent, and one of the most important stochastic gradient descent with momentum. This is all what we have discussed. In the next session, we are going to continue with the remaining, you know, optimizers. Thank you.